Foreword

Making software is hard, and it's difficult to know how to get better at it. There's always so much stuff to learn, and so many competing opinions about the right way to do things. And yet somehow here you are, about to read another book full of ideas you're apparently supposed to remember.

Well, the truth is that you can make software just fine without writing tests, and you can write tests just fine without using RSpec, so you'll be okay if you stop reading now. But if you decide to keep going, you'll discover something interesting: writing tests with RSpec is a great way to get really good at making software.

That's because RSpec isn't just a testing framework. It's a tool for learning how to think critically, patiently, and systematically about the design of your code, and how to make software in a methodical way so you have confidence that it's well organized, clear, and correct.

It's also the product of an ongoing experiment. Over the last decade or so, RSpec's users have refined the way they write programs and the strategies they use to test them, and RSpec itself has continually changed to reflect these emerging decisions. In recent years it's settled down into a stable, crystalline form: a kind of executable document of what all those programmers learned from their iterated successes and failures.

As a result, each feature in RSpec is there for a good reason. It was put there by a developer like you, to address a problem like the ones you face every time you write code. There's an art to appreciating this gallery of features as a unified whole and settling into the state of mind it wants you to adopt. You could spend months memorizing all of RSpec's methods and options and fiddly bits, yet still be none the wiser about how to use your tests to drive your implementation decisions and steer your design toward simplicity.

After all, the big challenge of test-driven development isn't knowing how to *write* tests, but knowing how to *listen* to them. For your tests to be worth the bytes they're written in, they must be able to speak to you about how well